RAG SYSTEM ENGINEERING KT GUIDE

Module: app/main.py (FastAPI Production API Layer)

Layer	Responsibility
Lifespan Layer	Preloads models during startup
FastAPI Layer	Creates API server
Request Validation Layer	Validates incoming requests
Normal API Layer	Returns JSON responses
Streaming Layer	Streams real-time LLM tokens

0. Learner Workflow

Step	What Learner Should Do
1	Open app/main.py
2	Understand FastAPI server architecture
3	Learn lifespan management
4	Trace model preloading flow
5	Understand request validation
6	Trace /ask endpoint
7	Learn streaming API architecture
8	Understand latency logging
9	Use FULL FILE REFERENCE for indentation clarity
10	Move to deployment modules

1. Position in Overall Architecture

```
Client / Frontend
↓
FastAPI Server
↓
main.py
↓
RAG Pipeline
↓
LLM Layer
↓
Streaming / JSON Response
```

2. Why main.py Exists

This module becomes the production API serving layer.

It solves:
• API exposure
• request validation
• streaming responses
• model preloading
• lifecycle management
• logging and latency tracking

3. Exact Step-by-Step Code Walkthrough

STEP 1 — Import FastAPI

```
from fastapi import FastAPI
```

Creates the API server layer.

STEP 2 — Import Pydantic

```
from pydantic import BaseModel
```

Provides request validation.

STEP 3 — Import StreamingResponse

```
from fastapi.responses import StreamingResponse
```

Enables real-time token streaming.

STEP 4 — Import asynccontextmanager

```
from contextlib import asynccontextmanager
```

Used for application lifecycle management.

STEP 5 — Import Internal Services

```
from app.services.pipeline import (
    run_rag,
    retrieve_context
)
```

Connects FastAPI to the RAG orchestration layer.

STEP 6 — Lifespan Function

```
@asynccontextmanager
async def lifespan(app: FastAPI):
```

Controls startup and shutdown lifecycle.

STEP 7 — Preload Models

```
init_vectorstore()
init_bm25()
```

Loads retrieval systems during startup.

STEP 8 — Create FastAPI App

```
app = FastAPI(lifespan=lifespan)
```

Attaches lifecycle manager to API server.

STEP 9 — Request Schema

```
class QueryRequest(BaseModel):
    query: str
```

Validates incoming request payloads.

STEP 10 — Root Endpoint

```
@app.get("/")
```

Health-check endpoint.

STEP 11 — Standard API Endpoint

```
@app.post("/ask")
```

Creates synchronous RAG API endpoint.

STEP 12 — Execute Full RAG Pipeline

```
result = run_rag(request.query)
```

Executes retrieval + generation pipeline.

STEP 13 — Streaming Endpoint

```
@app.post("/ask-stream")
```

Creates real-time streaming API.

STEP 14 — Streaming Generator

```
for token in generate_stream_answer(
    request.query,
    context
):
```

Streams LLM tokens progressively.

STEP 15 — StreamingResponse

```
return StreamingResponse(
    generate(),
    media_type="text/plain"
)
```

Returns live streamed tokens to client.

4. Production Engineering Insights

Concept	Why It Matters
Lifespan Hooks	Lower startup latency
Preloading	Faster inference
Pydantic Validation	Safer APIs
StreamingResponse	Better UX
Latency Tracking	Performance monitoring
Structured Logging	Observability
Health Endpoints	Deployment monitoring

5. FULL main.py FILE REFERENCE (Indentation Guide)

This section helps learners:

• understand FastAPI orchestration hierarchy

• manually implement the file correctly

• avoid indentation mistakes

• understand streaming flow deeply

```
from fastapi import FastAPI
from pydantic import BaseModel
from fastapi.responses import StreamingResponse
from contextlib import asynccontextmanager

import time
import logging
```

```
from app.services.pipeline import (
    run_rag,
    retrieve_context
)

from app.services.llm import (
    generate_stream_answer
)
```

```
# ====================================
# ■ LIFESPAN (STARTUP)
# ====================================

@asynccontextmanager
async def lifespan(app: FastAPI):

    from app.services.retriever import (
        init_vectorstore
    )

    from app.services.bm25_retriever import (
        init_bm25
    )

    print("■ Preloading models...")

    init_vectorstore()
    init_bm25()

    print("■ All models loaded")

    yield

    print("■ Server shutting down")
```

```
# ====================================
# ■ FASTAPI APP
# ====================================

app = FastAPI(lifespan=lifespan)


# ====================================
# ■ SETTINGS
# ====================================

ENABLE_LOGGING = True
ENABLE_LATENCY = True

logger = logging.getLogger("uvicorn.error")
```

```
# ====================================
# ■ REQUEST MODEL
# ====================================

class QueryRequest(BaseModel):
    query: str
```

```
# ====================================
# ■ ROOT
# ====================================

@app.get("/")
def read_root():

    return {
        "message": "RAG API is running"
    }
```

```
# ====================================
# ■ NORMAL RESPONSE
# ====================================

@app.post("/ask")
def ask_question(request: QueryRequest):

    if ENABLE_LOGGING:
        logger.info(f"Query: {request.query}")

    start = time.time()

    result = run_rag(request.query)

    end = time.time()
```

```
if ENABLE_LATENCY:

    latency = round(end - start, 2)

    print(f"Total Latency: {latency}s")

    logger.info(
        f"Total Latency: {latency}s"
    )

return {
    "query": request.query,
    "answer": result["answer"],
    "sources": result["sources"]
}
```

```
# ====================================
# ■ STREAMING RESPONSE
# ====================================

@app.post("/ask-stream")
def ask_stream(request: QueryRequest):

    def generate():

        if ENABLE_LOGGING:
            logger.info(f"Query: {request.query}")

        start = time.time()

        context, sources = retrieve_context(
            request.query
        )
```

```
    for token in generate_stream_answer(
        request.query,
        context
    ):

        yield token

    end = time.time()

    if ENABLE_LATENCY:

        latency = round(end - start, 2)

        print(
            f"Streaming Latency: {latency}s"
        )

        logger.info(
            f"Streaming Latency: {latency}s"
        )

return StreamingResponse(
    generate(),
    media_type="text/plain"
)
```

6. What To Learn Next

NEXT TOPICS:

- Dockerfile
- requirements.txt
- Deployment
- Redis caching
- Scaling strategies
- Production monitoring

These modules convert the RAG system into a production-grade enterprise deployment.